CS 432 – Databases

Track 1 – Assignment 4

Project: ShuttleGo – Shuttle Management and Booking System

Suhani 24110358 , Siju Niyati Samji 23110312 , Aeshaa Nehal Shah , K R Tanvi 23110149 , Makkena Lakshmi Manasa 23110193

Name	Roll Number
Niyati Siju	23110312
K R Tanvi	23110149
Makkena Lakshmi Manasa	23110193
Aeshaa Nehal Shah	23110018
Suhani	24110358

Github link: https://github.com/niyatiiii28/CS432_Track1_Assignment4

Video Link: Track1_assignment4_video.mp4

1. Introduction

ShuttleGo is a centralised shuttle management and booking system designed to address operational inefficiencies observed in traditional institutional and corporate shuttle services. The system manages members (passengers and drivers), vehicles, routes, trips, bookings, transactions, and associated records such as cancellations, no-show penalties, and driver assignments using a well-structured relational database.

In this assignment, the system was extended from a centralised database to a distributed one using data sharding. Instead of storing all data in a single database, it is now split across three MySQL shards to improve scalability and performance. The following sections document the shard key selection, partitioning strategy, query routing implementation, and a comprehensive scalability and trade-offs analysis.

2. Overview of Sharding

Data sharding is a technique where a database is divided into smaller parts (shards) and stored across multiple servers to improve scalability and performance. In this assignment, the ShuttleGo database is distributed across three MySQL shards, where each shard stores only a portion of the total data.

- Data is divided row-wise (horizontal partitioning)
- Each shard holds a subset of records
- The system decides which shard to use for each operation based on a hash function
- Some common undivided tables are replicated across all shards for convenient access

This setup helps distribute load and prepares the system for handling larger datasets efficiently.

SubTask 1: Shard Key Selection & Justification

3. Shard Key Selection

The shard key determines how data is distributed across different shards. For the ShuttleGo system, MemberID was chosen as the shard key because it satisfies all the required criteria:

High Cardinality

MemberID is unique for each user, ensuring a large number of distinct values. This helps in evenly distributing data across all shards and avoids overloading a single shard. Since every new registration generates a new, unique MemberID, the cardinality grows with the user base.

Query-Aligned

Most operations in the system — such as fetching a user's bookings, viewing their profile details, or checking their transaction history — are directly or indirectly linked to MemberID. This makes it efficient to locate and retrieve all related data from a single shard, minimising cross-shard operations.

Stable

MemberID is assigned at the time of user creation and does not change over time, ensuring consistent data placement without the need for redistribution. An unstable shard key would require costly data migrations whenever values change.

Thus, MemberID is a suitable shard key as it enables balanced data distribution, efficient query routing, and stable system behaviour.

4. Partitioning Strategy

A hash-based partitioning strategy was chosen for this system, where the shard is determined using the formula:

shard_id = MemberID % 3

This approach was selected for the following reasons:

- Uniform Distribution: Hash-based sharding spreads data evenly across all shards, avoiding data skew and overloaded shards.
- Balanced Data Distribution: New records are distributed across all shards equally instead of accumulating in one shard.
- Efficient for Our Queries: Most application queries are point lookups based on MemberID (e.g., fetching user details or bookings), and no critical queries are range-based, making hash-based routing more suitable than range-based distribution.
- Simple and Fast Routing: The shard can be computed instantly using a simple formula, making inserts and lookups straightforward with O(1) routing time.
- No Extra Metadata: It does not require maintaining a separate lookup table (unlike directory-based distribution), reducing overhead and avoiding a single point of failure.
- Scalable Design: As data grows, the distribution remains balanced without needing manual reorganisation.

5. Data Distribution, Scalability, and Skew Analysis

Using hash-based sharding on MemberID, the data is expected to be distributed almost equally across all three shards. The shard is decided using the formula MemberID % 3, so each new member created in the system is automatically assigned to one of the shards.

Shard	Port	MemberID Assignment Rule	Description
Shard 0	3307	MemberID % 3 == 0	Stores part of member data + all global tables
Shard 1	3308	MemberID % 3 == 1	Stores partitioned member data
Shard 2	3309	MemberID % 3 == 2	Stores partitioned member data

As new users keep registering, their data does not go to a single shard but is evenly spread across all shards, ensuring that the system scales smoothly without overloading any one shard. The risk of data imbalance is very low because distribution does not depend on value ranges or insertion

order. However, small variations can still occur since some members may have more related records (like bookings or transactions) than others. Overall, this imbalance is minimal and acceptable.

SubTask 2: Data Partitioning Implementation

6. Partitioning Model

The ShuttleGo system uses horizontal partitioning (sharding), where rows of large tables are distributed across multiple database shards. Each shard contains a subset of the data, and together they form the complete dataset.

Sharded Tables

The following tables are partitioned across shards based on MemberID:

Table	Shard Key	Notes
Member	MemberID	Primary user records; core of the shard routing
Passenger	MemberID (via PassengerID)	Co-located with Member on the same shard
Driver	MemberID	Co-located with Member on the same shard
Booking	PassengerID (derived from MemberID)	Routed via get_shard_by_passenger()
Transaction	MemberID (via BookingID)	Follows Booking to the same shard
BookingCancellation	MemberID (via BookingID)	Co-located with Booking
users	MemberID	Auth table co-located with Member
group_mappings	MemberID (via UserID)	RBAC mapping co-located with users

Global (Unreplicated) Tables

Certain tables are not partitioned and are instead stored in Shard 0 (port 3307) as global tables:

Table	Reason for Centralisation
Vehicle	Relatively small; referenced by all trips
Route	Small reference table; accessed across many operations
Trip	Contains seat availability; needs atomic updates
DriverAssignment	References Trip and Driver; kept with Trip
VehicleLiveLocation	Real-time data; centralised for consistency

These tables are relatively smaller and frequently accessed across different operations. Keeping them centralised simplifies query execution and avoids the need for cross-shard joins on reference data.

Data Co-location Strategy

To maintain relational consistency, all data related to a specific member is stored in the same shard. For example:

- A Member record and their Passenger/Driver profile
- Their Bookings and associated Transactions
- Their auth user record and group mappings

This ensures that most queries can be executed within a single shard, improving efficiency and avoiding expensive cross-shard joins.

Data Distribution Process

The partitioning process works as follows:

- A new record is assigned a unique MemberID (computed as global_max + 1 across all shards)
- The shard is computed: shard_id = MemberID % 3
- The record is inserted into the corresponding MySQL instance
- All related records (Passenger/Driver, users, group_mappings) are routed to the same shard in one committed transaction

7. Sharding Approach: Docker / Multiple Database Instances

Shard isolation was achieved by running three separate MySQL instances on the same host machine, each listening on a distinct port. This simulates the behaviour of physically separate database nodes:

Shard	Host	Port	Role
Shard 0	10.0.116.184	3307	Primary shard + global tables
Shard 1	10.0.116.184	3308	Secondary shard
Shard 2	10.0.116.184	3309	Tertiary shard

Each shard uses the same database schema (same_team_name database), and shard isolation is enforced entirely at the application layer — each shard connection is obtained via get_connection(port), and queries are never broadcast to all shards unless explicitly implementing a fan-out pattern.

SubTask 3: Query Routing Implementation

8. Routing Architecture

Query routing is implemented directly in the Flask application layer (app.py). Every API request extracts the shard key (MemberID) and computes the target shard using the hash formula, directing the query to the correct MySQL instance.

Layer	Component	Description
1	Client (Browser / API)	User interacts with the system via UI or API requests
2	Flask Backend	Handles API requests, validation, and business logic
3	Shard Router	Computes shard_id = MemberID % 3 and selects the connection
4	Database Shards	Three MySQL instances on ports 3307, 3308, 3309

9. Three Routing Patterns

Pattern 1: Single-Key Lookup Routing

For point lookups where the MemberID is known, the application calls get_shard(member_id) which returns the exact connection for that shard. This avoids unnecessary cross-shard overhead.

Examples:

- GET /api/members/<mid> — computes mid % 3 and queries only that shard
- GET /api/me — reads the session's member_id and routes directly to the correct shard
- Booking lookups — get_shard_by_passenger(pid) routes by PassengerID since PassengerID is derived from MemberID and follows the same distribution

Pattern 2: Insert Routing

On registration (POST /api/register), the application:

- Queries all three shards to determine the global maximum MemberID
- Increments it to get the new ID: new_mid = max_mid + 1
- Calls get_shard(new_mid) to route all related inserts (Member, Passenger/Driver, users, group_mappings) to the single correct shard in one committed transaction

Pattern 3: Range Queries (Fan-out and Merge)

For queries that cannot be resolved to a single shard — such as date range bookings or admin-level listing — the application fans out to all three shards:

- Fan out: execute the same query on each shard in sequence
- Collect results from all three shards
- Merge into a single sorted list (by TripDate and BookingTime)
- Remove duplicate BookingIDs that can arise from replicated Trip/Route data
- Return a shard_breakdown field showing how many rows each shard contributed

Admin endpoints such as GET /api/bookings and GET /api/admin/users also use this scatter-gather pattern since the target MemberID is not known in advance.

Global Table Access

Tables that are not sharded — Vehicle, Route, Trip, VehicleLiveLocation, DriverAssignment — are always accessed via get_db(), which returns the Shard 0 connection (port 3307). This avoids cross-shard joins for trip and route data while keeping those reads fast and consistent.

10. Experiments & Testing

10.1 ACID & Reliability Tests

Test	Description	Result
Atomicity	Simulated failure during insert; verified no partial data remains	PASS
Consistency	Performed multiple concurrent updates; checked index matches DB state	PASS
Durability	Committed transactions, restarted system; verified data persisted	PASS
Crash Recovery	Forced crash during execution; system recovered using WAL logs	PASS
Rollback Testing	Triggered failure manually; verified all changes were undone	PASS

Key observations: SQLite WAL and MySQL transaction management play a critical role in ensuring reliability. B+ Tree index synchronisation is essential for correctness, and the recovery mechanism successfully prevents data corruption.

10.2 Concurrent Workload & Stress Tests

Test 1: Race Condition — 20 Concurrent Bookings on Same Seat

Metric	Value
Total Requests	20 concurrent threads
Successful (2xx)	1
Blocked (400/409)	19
Elapsed Time	0.035s
Result	PASS — Isolation constraint correctly prevented double-booking

Despite 20 users racing for the same seat simultaneously, the database correctly serialised the transactions. Only the first request was committed; the subsequent 19 were identified as conflicts

and blocked. This proves the system's locking mechanisms and unique constraints prevent double-booking errors.

Test 2: Stress Test — 300 Rapid Concurrent Requests

Metric	Value
Total Requests	300 (batches of 50 threads)
HTTP 200 OK	300
Errors	0 (0% error rate)
Total Time	34.16s
Throughput	8.8 req/s
Response Time (Median)	2358.3 ms
Response Time (Mean)	3139.6 ms
Response Time (Min)	617.2 ms
Response Time (Max)	5940.7 ms
Result	PASS — Strong consistency maintained under heavy load

Even under a heavy load that pushed response times up to nearly 6 seconds, the error rate remained at 0%. Every request eventually received a valid, correct response, demonstrating that the system is stable and reliable under significant concurrent pressure.

Test 3: Failure Simulation — Abrupt Client Crashes Mid-Request

Metric	Value
Crash Requests Fired	10 (timeout after 0.05s)
Simulates	Client crash / network drop before response
DB Health Check	HTTP 200 (Success)
Post-Crash State	System remained responsive
Result	PASS — Atomicity confirmed; no partial data persisted

Because the database responded correctly after the crashes, SQLite's Write-Ahead Logging (WAL) and the Flask transaction manager successfully rolled back any partial data. Half-finished bookings were not saved, ensuring the database never entered a corrupted state.

Test 4: Durability — Write-Then-Read Verification

Metric	Value
Booking POST Status	HTTP 201 (Created)
Read-back GET Status	HTTP 200 (OK)

Metric	Value
Bookings Visible	4 (including the newly created Seat 40 booking)
Result	PASS — Committed data immediately visible in separate read request

The fact that the newly created booking was immediately visible in a separate read request proves that once a transaction is committed, it is permanently stored. This satisfies the durability requirement of ACID compliance.

SubTask 4: Scalability & Trade-offs Analysis

11. Horizontal vs. Vertical Scaling

In the implemented system, horizontal scaling is achieved by distributing data across multiple database shards running on different ports (3307, 3308, 3309). A hash-based partitioning strategy assigns data to a shard based on:

Shard ID = MemberID % 3

Dimension	Horizontal Scaling (Our Approach)	Vertical Scaling
How it works	Distribute data across multiple DB nodes	Upgrade CPU, RAM, storage on one server
Scalability limit	Scales out by adding more shards	Limited by hardware ceiling
Cost	Commodity hardware, cost-effective	Diminishing returns at scale
Our implementation	User queries → single shard; admin → all shards	Not used
Trade-off	Complexity in routing and consistency	Simpler but limited

In our implementation, user-specific queries are routed to a single shard using deterministic hashing, while administrative queries aggregate data by scanning all shards. Certain global tables (such as Trip) are stored in a default shard. Horizontal scaling therefore improves system scalability and load distribution, whereas vertical scaling would be limited by hardware constraints and becomes inefficient at large scale.

12. Consistency

Consistency in this system is maintained at the shard level through deterministic routing. Since each record is always mapped to the same shard using the hash function, read and write operations for a given user are performed on the same shard, guaranteeing strong consistency for single-user operations.

However, consistency challenges arise in scenarios involving cross-shard operations:

- Login and administrative queries require searching across multiple shards simultaneously
- Booking operations involve updates to both a shard (Booking table) and a global table (Trip for seat availability) — these two writes are not atomic across the two connections
- If the booking insert succeeds but the Trip seat decrement fails, the data across shard and global table becomes temporarily inconsistent

Stress testing results indicate that concurrency control is effective at the shard level, as race conditions (e.g., multiple users attempting to book the same seat) are properly handled through unique constraints and MySQL locking.

Conclusion: Strong consistency within individual shards; weaker consistency guarantees across shards, particularly for cross-shard booking + global table updates.

13. Availability

Availability refers to the system's ability to continue functioning despite failures. In the current design:

- If one shard becomes unavailable, only the data stored in that shard is affected — the remaining two shards continue to function normally
- This results in partial availability: the system remains operational for users whose data resides on unaffected shards
- Users whose MemberID hashes to the failed shard will be unable to log in, view bookings, or make new bookings
- The system does not implement replication or redundancy mechanisms — there is no automatic failover
- Global tables (on Shard 0) represent a single point of failure: if Shard 0 goes down, trip data and seat availability become inaccessible to all users

Conclusion: The system offers partial availability. Stress testing confirms stable operation under normal concurrent load, but shard failures are not gracefully handled.

14. Partition Tolerance

Partition tolerance refers to the system's ability to continue functioning when there is a communication failure between different parts of the system. In this implementation:

- There is no replication or backup mechanism across shards
- Requests are routed strictly based on the shard mapping formula, with no fallback strategy
- If a shard becomes unreachable due to a network failure, requests mapped to that shard will fail — the system does not attempt recovery or rerouting
- Failure simulation tests confirm that client-side failures do not corrupt the database (transactions remain atomic via WAL), but shard-level network partitions are not tolerated

Conclusion: The system has limited partition tolerance. It prioritises simplicity and consistency over fault tolerance.

15. CAP Theorem Analysis

Based on the analysis above, the ShuttleGo sharding implementation can be characterised as follows with respect to the CAP theorem:

CAP Property	Status	Explanation
Consistency (C)	Strong (within shard) / Weak (cross-shard)	Strong for per-user operations; weaker for cross-shard booking + Trip updates
Availability (A)	Partial	Depends on which shard is down; global table failure is system-wide
Partition Tolerance (P)	Weak	No replication or failover; shard failure causes data loss for that partition

16. Observations & Limitations

Observations

- Hash-based sharding is highly effective for evenly distributing member-centric data
- Query routing is critical — incorrect routing would silently return empty results rather than errors
- Data co-location significantly improves per-user query performance by avoiding cross-shard joins
- WAL-based logging and MySQL transactions ensure reliability and atomicity at the shard level
- The fan-out pattern for admin queries introduces latency proportional to the number of shards
- The stress test demonstrates that the system handles 300 concurrent requests with 0% error rate, though response times degrade under load

Limitations

- Range queries require fan-out across all shards — there is no way to know which shards contain relevant data for date-range queries without querying all
- Global uniqueness checks (e.g., checking if a username already exists) require scanning all shards, adding latency to registration
- No native cross-shard joins — joins that span sharded and global tables must be performed in application code
- Global tables (Trip, Route, Vehicle) represent a single point of failure centralised on Shard 0
- No automatic rebalancing — if the number of shards changes, all data would need to be redistributed
- The current global-max-ID approach for generating new MemberIDs creates a bottleneck during high-concurrency registration

17. Conclusion

The ShuttleGo system was successfully extended from a centralised database to a distributed one using horizontal sharding. By selecting MemberID as the shard key and applying hash-based partitioning (shard_id = MemberID % 3), the system achieves balanced data distribution, efficient query routing, and a scalable architecture across three MySQL shard instances.

The implementation demonstrates core distributed database concepts including horizontal scaling, data locality, fan-out queries, and the fundamental trade-offs described by the CAP theorem. Experimental results confirm that the system maintains strong consistency and atomicity within shards, handles concurrent bookings correctly through database-level locking, and remains stable under significant load (300 concurrent requests, 0% error rate).

The key trade-offs introduced by sharding — increased system complexity, weaker cross-shard consistency, partial availability, and limited partition tolerance — are well-understood and reflect the real-world challenges of distributed database design. Future improvements could include replica sets for each shard, a distributed transaction coordinator for cross-shard operations, and consistent hashing to enable dynamic shard addition without full data redistribution.